Greedy Scheduling and Divide-and-Conquer FFT

Pradhyuman Singh Shekhawat
University of Florida
COT5405 — Analysis of Algorithms
Fall 2026

I. Introduction

This project studies two algorithmic techniques through separate application problems. The first is a greedy algorithm that selects a maximum-cardinality set of mutually compatible communication intervals from a fixed candidate set. The second is a divide-and-conquer radix-2 Cooley–Tukey Fast Fourier Transform (FFT) for computing the discrete Fourier transform of sampled vibration signals.

For each component, the report defines the application and mathematical problem, describes the algorithm, analyzes its running time, proves correctness, and presents implementation and empirical validation. The scheduling result is optimal only for the supplied fixed interval set; it does not solve the full multi-resource Deep Space Network scheduling problem. The FFT computes a spectrum and is not itself a bearing-fault classifier.

II. Greedy Algorithm: Communication-Interval Scheduling

A. Application and Problem Definition

Deep-space communication planning must respect antenna availability and non-overlapping use of a resource. This project uses candidate communication intervals associated with antenna DSS-34. For the algorithmic abstraction, each candidate interval has a start time and finish time, and all intervals have equal value. The objective is to select the largest number of pairwise non-overlapping intervals.

Intervals are interpreted as half-open intervals $[s_i, f_i)$; therefore, one interval ending exactly when another begins is compatible. This is a simplified single-resource abstraction, not a complete model of operational Deep Space Network scheduling.

B. Mathematical Abstraction

Let $I = \{1, \ldots, n\}$ be the set of candidate intervals. Interval i has start time s_i and finish time f_i, where $s_i < f_i$. A feasible set $S \subseteq I$ satisfies, for every distinct pair $i, j \in S$, either $f_i \leq s_j$ or $f_j \leq s_i$. The objective is

$$\max |S| \quad \text{subject to } S \text{ being feasible.}$$

The unit-value assumption is important: maximizing the number of intervals does not necessarily maximize total communication duration or scientific value.

C. Algorithm

Sort intervals in nondecreasing order of finish time. Select the first interval, then scan the remaining sorted intervals and select interval i whenever $s_i \geq f_{\text{last}}$, where f_{last} is the finish time of the most recently selected interval.

Listing 1: Earliest-finish-time greedy algorithm (pseudocode)

```
sort intervals by nondecreasing finish time

selected = []
last_finish = -infinity

for interval in intervals:
    if interval.start >= last_finish:
        selected.append(interval)
        last_finish = interval.finish

return selected
```

D. Correctness Proof

Let g be an interval with the earliest finish time among all available intervals. Let O be an optimal solution, and let o be its first interval. Since $f_g \leq f_o$, replacing o with g preserves feasibility for every later interval in O: any interval that starts at or after f_o also starts at or after f_g. Thus, there exists an optimal solution that begins with g.

After selecting g, the remaining compatible intervals all start at or after f_g. The same argument applies to this remaining subproblem. Repeating the exchange argument proves that each greedy choice can be part of an optimal solution, so the algorithm returns a maximum-cardinality feasible subset.

This proof applies to the fixed candidate intervals and the unit-value objective defined above, not to arbitrary weighted scheduling or the full DSN problem.

E. Running-Time Analysis

Sorting takes $O(n \log n)$ time. The subsequent scan takes $O(n)$ time, so the total running time is $O(n \log n)$, dominated by sorting. Storing the sorted intervals and selected set requires $O(n)$ space in the implementation.

F. Audit and Scheduling Results

The audit pipeline reported 1,452 requests and 874 candidate intervals for DSS-34, corresponding to 60.193% of the requests. The scheduler selected 119 intervals and skipped 755 conflicting candidates. The selected interval counts by week were W10: 27, W20: 21, W30: 24, W40: 24, and W50: 23.

These figures describe the current pipeline output and should not be interpreted as a global optimum over every possible visibility opportunity. The greedy algorithm is optimal on the fixed candidate interval set supplied to it, under the maximum-cardinality objective.

G. Correctness and Feasibility Validation

The implementation was tested using unit tests and an independent optimality checker. The checker compared the greedy algorithm's selection count with a dynamic-programming solution on 1,000 randomly generated small interval sets. The corrected checker reported matching optimal values for those test instances.

A feasibility check of the selected schedule reported zero overlap violations among the 119 selected intervals. These tests provide evidence for the implementation's correctness on the tested inputs; they do not replace the mathematical correctness proof or establish optimality for the full operational DSN scheduling problem.

H. Greedy Runtime Measurements

The repository contains runtime benchmark artifacts under `greedy_dsn/results/runtime/`. The CSV file and plot generated by the final benchmark run should be retained with the project.

TABLE I: Greedy scheduler median runtime (seven repetitions).

Input size n	Median runtime (ms)
100	0.037130
250	0.110246
500	0.278891
1,000	0.501851
2,000	1.280528
4,000	3.389668
8,000	6.338986

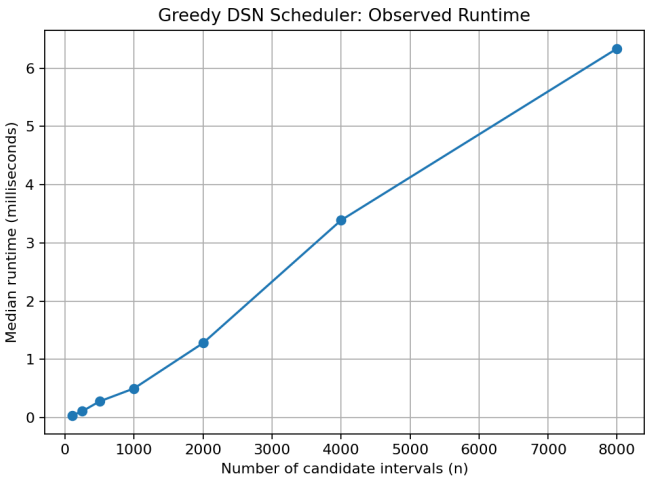


Fig. 1: Measured runtime of the greedy scheduling implementation.

The expected asymptotic running time is $O(n \log n)$, primarily because the intervals must be sorted. The benchmark plot provides empirical evidence about the implementation on the tested machine. Observed timings can vary because of interpreter overhead, system load, and input characteristics.

III. Divide-and-Conquer Algorithm: Bearing-Vibration FFT

A. Application and Scope

Vibration sensors record sampled time-domain signals from rotating machinery. Transforming these samples into the frequency domain helps engineers inspect prominent frequency components. The Case Western Reserve University Bearing Data Center provides documented bearing vibration measurements and experimental conditions [1].

An FFT is a signal-processing step only: a spectrum does not by itself diagnose a bearing fault or classify its type.

B. Mathematical Abstraction

Given a sequence $x[0], \ldots, x[n-1]$ with $n = 2^k$, compute its discrete Fourier transform (DFT):

$$X[m] = \sum_{j=0}^{n-1} x[j] e^{-2\pi i j m/n}, \qquad m = 0, \ldots, n-1.$$

Here $x[j]$ is a time-domain sample and $X[m]$ is the complex coefficient at frequency bin m. For sample rate f_s, the bin spacing is f_s/n Hz.

C. Divide-and-Conquer Algorithm

Separate the input into even- and odd-indexed subsequences: $x_e[r] = x[2r]$ and $x_o[r] = x[2r+1]$. Recursively compute their length-$n/2$ DFTs, denoted by $E[m]$ and $O[m]$. With $W_n^m = e^{-2\pi i m/n}$, combine:

$$X[m] = E[m] + W_n^m O[m],$$

$$X[m + n/2] = E[m] - W_n^m O[m],$$

for $0 \le m < n/2$. The base case is a one-element sequence. The implementation rejects empty input and lengths that are not powers of two.

D. Correctness Proof

Split the DFT sum into even and odd indices:

$$X[m] = \sum_{r=0}^{n/2-1} x[2r] e^{-2\pi i (2r) m/n} + \sum_{r=0}^{n/2-1} x[2r+1] e^{-2\pi i (2r+1) m/n}.$$

The first sum is $E[m]$. Factoring $e^{-2\pi i m/n}$ from the second sum makes it $W_n^m O[m]$, hence

$$X[m] = E[m] + W_n^m O[m].$$

For the coefficient at $m + n/2$, the even-index exponential is unchanged, while the odd-index exponential acquires a factor of $e^{-\pi i} = -1$. Therefore,

$$X[m + n/2] = E[m] - W_n^m O[m].$$

For $n = 1$, the algorithm returns the input, which is its DFT. Assume the recursive calls correctly compute the DFTs of length $n/2$. The derived combination equations then produce all n DFT coefficients correctly. Induction on powers of two proves correctness for every accepted input length.

E. Running-Time Analysis

At each recursion level, two subproblems of size $n/2$ are solved and the combination step performs $\Theta(n)$ work. Thus,

$$T(n) = 2T(n/2) + \Theta(n), \qquad T(1) = \Theta(1).$$

By the Master Theorem, $T(n) = \Theta(n \log n)$. This recursive implementation uses $O(n)$ auxiliary space across its lists and recursive computations.

F. Correctness Tests and Synthetic Demonstration

Seven unit tests passed. They check the one-sample base case, constant and impulse signals, a sinusoid with a known frequency bin, comparison with a direct DFT on deterministic random inputs, and rejection of invalid input lengths.

The synthetic demonstration used $n = 1024$ samples at $f_s = 1024$ Hz, giving 1 Hz bin spacing. The largest magnitudes appeared at 64 Hz, 180 Hz, and 230 Hz, as expected from the generated signal. These results validate the implementation on controlled inputs; they are not results from a real bearing recording.

G. Empirical Runtime Results

Table II reports the measured median runtime from seven repetitions on the tested machine. These are synthetic-input timings.

TABLE II: Radix-2 FFT median runtime (seven repetitions).

Input size n	Median runtime (ms)
256	0.551136
512	1.375953
1,024	2.468382
2,048	5.763517
4,096	11.238437
8,192	25.909256
16,384	55.074267
32,768	120.434025

The measured runtime increases with input size and broadly follows the expected $n \log n$ growth. Deviations from the scaled reference curve can arise from Python recursion and list-allocation overhead, interpreter behavior, and machine load. The recurrence, rather than the timing plot, establishes the asymptotic bound.

H. Real-Data Demonstration

The FFT implementation was also evaluated on a vibration recording from the Case Western Reserve University Bearing Data Center [1]. The input file was `data/105.mat`, and the selected channel was `X105_DE_time`. The first 4,096 samples were used with a sampling rate of 12,000 Hz. The mean was removed from the segment before computing the spectrum.

The resulting frequency resolution was

$$\Delta f = \frac{f_s}{n} = \frac{12000}{4096} \approx 2.93 \text{ Hz}.$$

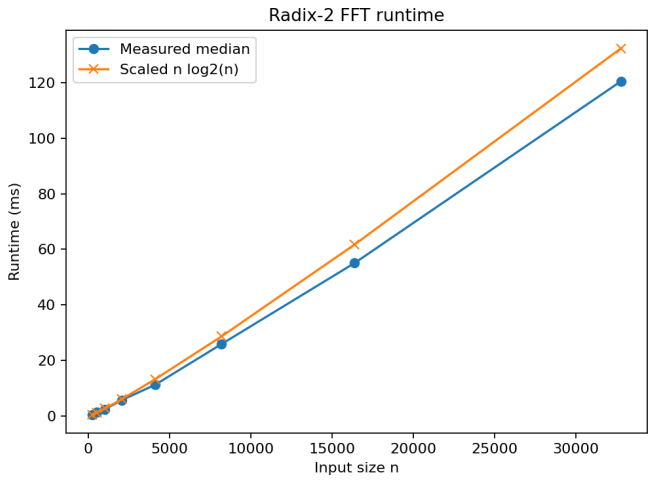


Fig. 2: Measured FFT runtime compared with a scaled $n \log_2 n$ curve.

The largest reported spectral magnitudes occurred near 3585.94 Hz, 2616.21 Hz, and 2879.88 Hz. These values describe peaks in the processed segment; they are not, by themselves, evidence of a specific bearing defect.

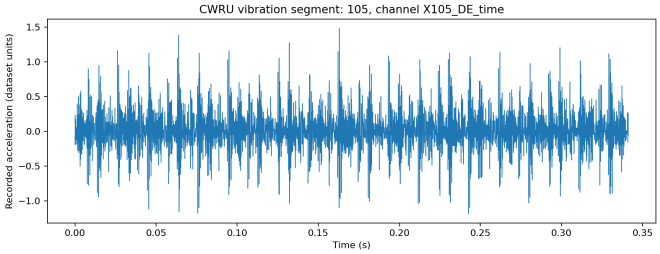


Fig. 3: Time-domain waveform for the selected segment of channel `X105_DE_time`.

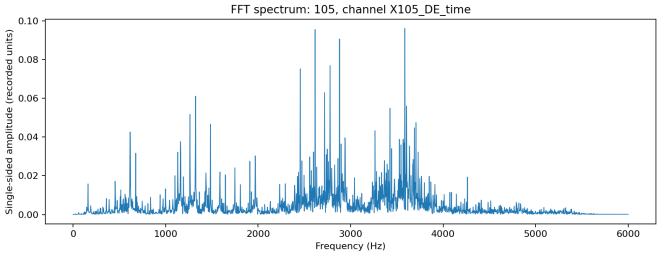


Fig. 4: FFT magnitude spectrum of the selected real vibration segment.

The synthetic benchmark and real-data demonstration serve different purposes. The synthetic signal provides known frequency components for checking expected behavior, while the real recording demonstrates processing of an experimental vibration signal. The spectrum is an analytical output, not a diagnosis or a classification result.

IV. Comparison and Discussion

The greedy algorithm makes locally optimal choices supported by an exchange argument and solves a maximum-cardinality interval-selection problem in $O(n \log n)$ time. The FFT recursively decomposes the DFT and combines partial solutions using roots of unity, achieving $O(n \log n)$ time.

Although both bounds share the same asymptotic form, they describe different operations and input models. Their empirical results should not be compared directly because the measured workloads and units of work differ.

The greedy algorithm's correctness guarantee depends on the fixed-candidate, single-resource, equal-value interval abstraction. The FFT's correctness guarantee applies to accepted power-of-two input lengths and follows from the DFT decomposition and induction. In both cases, theoretical analysis and empirical testing play complementary roles.

V. Limitations and Future Work

The scheduling model assumes a fixed candidate interval set, a single antenna, and equal value per interval. It omits global optimization over all visibility opportunities and broader operational constraints. Future work could examine more realistic resource constraints, multiple antennas, and objectives that account for communication duration or scientific value. Such extensions would require a separate algorithmic formulation and correctness analysis.

The FFT implementation uses radix-2 recursion and therefore accepts only input lengths that are powers of two. The benchmark measures execution time on one machine, and the observed timings may vary across environments. Future work could compare the implementation against a trusted numerical library and examine larger or differently structured signals.

The real-data demonstration uses one selected channel and one segment. Its spectral peaks should not be treated as a bearing diagnosis. Interpreting the condition of machinery would require additional domain-specific analysis and supporting evidence.

VI. Use of AI Tools

Generative AI tools were used during the development of this project to support algorithm design, implementation, debugging, test development, and the revision of the written report. All algorithmic analyses, correctness arguments, experimental results, and conclusions remain the responsibility of the author and were reviewed against the project implementation and test results.

The project archive available for the final review did not include a complete record of the earlier AI-assisted development sessions. Therefore, the original prompts, responses, and intermediate outputs from those sessions could not be reproduced in full. The available record is documented in

`docs/AI_USE_LOG.md`. No attempt has been made to reconstruct missing exchanges or present reconstructed material as an exact transcript.

The verification evidence summarized below is drawn from the retained source code, test results, benchmark outputs, and report-build artifacts. It documents the checks performed on the submitted work but does not replace the complete prompt-and-response record required by the course. Any remaining gaps in that record must be addressed using the original conversation history.

VII. AI Tool Use and Prompt Log

A. Purpose and disclosure method

ChatGPT was used as a supporting tool for project planning, algorithm-design discussion, implementation assistance, testing strategy, and report preparation. The student remains responsible for the correctness of mathematical formulations, proofs, code, citations, and experimental claims. AI-generated suggestions were treated as proposals requiring review, not as independent evidence.

This log covers the selected project: (i) greedy communication scheduling motivated by NASA Deep Space Network (DSN) operations, and (ii) divide-and-conquer analysis of bearing-vibration signals using a radix-2 Cooley–Tukey Fast Fourier Transform (FFT). Earlier provisional examples, such as generic activity selection or closest-pair geometry, are not presented as the final project implementations.

The original project-planning prompt is retained in the project conversation/source text dated 2026-10-12. Where exact prompt wording is available, it is quoted below. Where the retained record contains only a summary, that limitation is explicitly identified rather than presenting reconstructed wording as a verbatim prompt. Historical model-version metadata was not preserved for every interaction; this appendix therefore does not assign an unverified model version to earlier turns.

B. AI-Assisted Project Development

1) **Project requirements and problem selection.**
The initial planning focused on identifying suitable real-world problems for the greedy and divide-and-conquer components and evaluating their mathematical and practical suitability.
   - **Prompt:**
     > "Act as my algorithm-design tutor, research assistant, project supervisor, and implementation partner. Help me select suitable problems, design and prove algorithms, implement them, conduct experiments, and produce a professional research-style report. I want to work through the project with you step by step. Do not jump directly into writing code or a report."
   - **Outcome:** Candidate problems were compared based on their real-world motivation, mathematical formulation, suitability for the required algorithm, correctness guarantees, data availability, and experimental feasibility.
   - **Refinement:** The project was organized into stages covering problem definition, mathematical modeling, algorithm design, correctness proofs, complexity analysis, implementation, testing, benchmarking, and report preparation. The selected models were restricted to problems whose assumptions and algorithmic guarantees could be stated precisely.
2) **Greedy scheduling model.**
The greedy component was developed around a satellite communication scheduling problem inspired by NASA Deep Space Network operations.
   - **Outcome:** The problem was formulated as selecting the maximum number of pairwise non-overlapping intervals from a fixed set of candidate communication opportunities. The earliest-finish-time greedy algorithm was selected for this model.
   - **Refinement:** The model was restricted to a single resource and equal-value intervals. It does not claim to solve the complete operational DSN scheduling problem or account for all its operational constraints.
3) **Greedy implementation and testing.**
   - **Recorded results:** The project progress records report 874 candidate requests and 119 selected intervals. They also report ten unit tests and comparisons against a dynamic-programming solution on 1,000 randomly generated instances.
   - **Verification:** These results should be confirmed against the final source code, test output, and saved experiment files before being presented as final experimental findings.
4) **Divide-and-conquer problem formulation.**
The second component uses the radix-2 Cooley–Tukey Fast Fourier Transform (FFT) to compute the frequency spectrum of sampled bearing-vibration signals.
   - **Outcome:** The input was defined as a sequence of $n = 2^k$ samples, and the required output as its discrete Fourier transform:
     $$X[m] = \sum_{j=0}^{n-1} x[j] e^{-2\pi i j m/n}.$$
     The algorithm divides the input into even- and odd-indexed subsequences, transforms them recursively, and combines their results.
   - **Complexity analysis:** The recurrence
     $$T(n) = 2T(n/2) + \Theta(n)$$
     gives a time complexity of $\Theta(n \log n)$.
   - **Refinement:** The application was limited to frequency-spectrum computation using bearing-vibration data associated with the Case Western Reserve University (CWRU) Bearing Data Center.

The FFT was not treated as a complete bearing-fault diagnosis or classification system.

5) **FFT implementation and experiments.**
   - **Recorded results:** The project progress records report seven FFT tests, recovery of synthetic signal frequencies at 64, 180, and 230 Hz, and runtime experiments with input sizes ranging from 256 to 32,768 samples, using seven repetitions.
   - **Verification:** The reported results should be checked against the final implementation, test output, benchmark data, and generated plots. Synthetic-signal tests and experiments using real bearing measurements should be treated as separate forms of validation.
6) **Report review and reproducibility.**
   The report was reviewed against the course rubric, focusing on correctness proofs, complexity analysis, reproducibility, citations, and consistency between the report and implementation.
   - **Prompt:**

     "Check the following:

     a) Correctness proofs: verify that the report contains a valid proof for each algorithm, not just a complexity explanation or test results.
     b) Reproducibility: make sure the documented commands work from the stated directory without undocumented environment setup.
     c) Full report-to-code consistency: verify the reported results, tables, figures, citations, and data descriptions against the actual files.

     "
   - **Outcome:** The review focused on the greedy exchange argument, the FFT decomposition, test execution, benchmark results, citations, and report structure. The retained project notes also record changes to the standalone greedy verifier, a counterexample test, and the use of the IEEE conference template.
   - **Refinement:** The review identified items requiring correction or further checking. Final claims were to be checked against the project files and reproducible results.

C. Verification and Responsibility

AI assistance supported project planning, explanations, implementation, testing, and report review. I am responsible for the final submission. The mathematical proofs must match the stated problem formulations, experimental results must be reproducible from the included code, and claims in the report must be supported by the corresponding evidence and citations.

The reported test counts and benchmark results are based on retained project records and should be confirmed against the final source code and experiment outputs. Any result that cannot be reproduced should be corrected or removed.

APPENDIX A
RUNTIME-VALIDATION CODE

The benchmark programs used to collect the empirical running-time measurements are included below. The corresponding CSV files and plots are retained in the project directory. These measurements depend on the machine and Python environment; the asymptotic bounds follow from the analyses in the main text.

A. Greedy interval-scheduling benchmark

Listing 2: Greedy runtime benchmark source.

```python
#!/usr/bin/env python3
"""Reproducible runtime benchmark for greedy interval scheduling."""
import argparse
import csv
import random
import statistics
import sys
import time
import math
from pathlib import Path

PROJECT_ROOT = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(PROJECT_ROOT / "src"))

try:
    from greedy_satnet import greedy_schedule
except ImportError as exc:
    raise SystemExit(
        "Could not import greedy_satnet.py from greedy_dsn/src/."
    ) from exc


def make_intervals(n, seed):
    """Generate reproducible positive-duration synthetic intervals."""
    rng = random.Random(seed)
    intervals = []
    for i in range(n):
        start = rng.randint(0, max(100, n * 10))
        duration = rng.randint(1, 100)
        intervals.append({
            "week_key": "SYNTHETIC",
            "track_id": str(i),
            "activity_start_epoch": str(start),
            "activity_end_epoch": str(start + duration),
        })
    return intervals


def median_runtime_ms(intervals, repeats):
    samples = []
    for _ in range(repeats):
        t0 = time.perf_counter()
        result = greedy_schedule(intervals)
        elapsed = time.perf_counter() - t0
        samples.append(elapsed * 1000.0)
        assert len(result) <= len(intervals)
    return statistics.median(samples), min(samples), max(samples)


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument(
        "--sizes",
        default="100,250,500,1000,2000,4000,8000",
        help="Comma-separated input sizes",
    )
    parser.add_argument("--repeats", type=int, default=7)
    parser.add_argument("--seed", type=int, default=5405)
    parser.add_argument("--out", default=str(PROJECT_ROOT / "results" / "runtime"))
    args = parser.parse_args()

    sizes = [int(x.strip()) for x in args.sizes.split(",") if x.strip()]
    if not sizes or any(n < 1 for n in sizes):
        parser.error("--sizes must contain positive integers")
    if args.repeats < 3:
        parser.error("--repeats must be at least 3")

    outdir = Path(args.out)
    outdir.mkdir(parents=True, exist_ok=True)
    results = []

    print("GREEDY INTERVAL-SCHEDULING RUNTIME BENCHMARK")
    print("Timer covers greedy_schedule() only; input generation is excluded.")
    print(f"Seed: {args.seed}; repeats per size: {args.repeats}")
    print("n, median_ms, min_ms, max_ms, n_log2_n")
```

```
    for n in sizes:
        intervals = make_intervals(n, args.seed + n)
        median_ms, min_ms, max_ms = median_runtime_ms(intervals, args.repeats)
        row = {
            "n": n,
            "median_ms": f"{median_ms:.6f}",
            "min_ms": f"{min_ms:.6f}",
            "max_ms": f"{max_ms:.6f}",
            "n_log2_n": f"{n * math.log2(n):.6f}",
            "seed_used": args.seed + n,
            "repeats": args.repeats,
        }
        results.append(row)
        print(
            f"{n}, {row['median_ms']}, {row['min_ms']}, "
            f"{row['max_ms']}, {row['n_log2_n']}"
        )

    csv_path = outdir / "runtime_results.csv"
    with csv_path.open("w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=list(results[0].keys()))
        writer.writeheader()
        writer.writerows(results)

    try:
        import matplotlib.pyplot as plt
    except ImportError:
        print("matplotlib is not installed; CSV saved, but graphs were not created."
      )
        print("Install with: python3 -m pip install matplotlib")
        print("CSV:", csv_path.resolve())
        return

    ns = [int(r["n"]) for r in results]
    medians = [float(r["median_ms"]) for r in results]
    nlogn = [float(r["n_log2_n"]) for r in results]

    fig, ax = plt.subplots()
    ax.plot(ns, medians, marker="o")
    ax.set_xlabel("Number of candidate intervals (n)")
    ax.set_ylabel("Median runtime (milliseconds)")
    ax.set_title("Greedy DSN Scheduler: Observed Runtime")
    ax.grid(True)
    fig.tight_layout()
    runtime_plot = outdir / "runtime_median.png"
    fig.savefig(runtime_plot, dpi=160)
    plt.close(fig)

    scale = medians[0] / nlogn[0] if nlogn[0] else 1.0
    reference = [value * scale for value in nlogn]

    fig, ax = plt.subplots()
    ax.plot(ns, medians, marker="o", label="Observed median runtime")
    ax.plot(ns, reference, marker="s", label="Scaled n log2(n) reference")
    ax.set_xlabel("Number of candidate intervals (n)")
    ax.set_ylabel("Runtime / scaled reference (milliseconds)")
    ax.set_title("Observed Runtime vs. O(n log n) Growth Reference")
    ax.legend()
    ax.grid(True)
    fig.tight_layout()
    comparison_plot = outdir / "runtime_vs_nlogn.png"
    fig.savefig(comparison_plot, dpi=160)
    plt.close(fig)


    # Log-log plot to examine runtime growth across input sizes.
    fig, ax = plt.subplots()
    ax.plot(ns, medians, marker="o", label="Observed median runtime")
    ax.set_xscale("log")
    ax.set_yscale("log")
    ax.set_xlabel("Number of candidate intervals (n, log scale)")
    ax.set_ylabel("Median runtime (milliseconds, log scale)")
    ax.set_title("Greedy DSN Scheduler: Log-Log Runtime")
    ax.legend()
    ax.grid(True, which="both", linestyle="--", linewidth=0.5)
    fig.tight_layout()

    loglog_plot = outdir / "runtime_loglog.png"
    fig.savefig(loglog_plot, dpi=160)
    plt.close(fig)

    print("CSV:", csv_path.resolve())
    print("Plot:", runtime_plot.resolve())
    print("Plot:", comparison_plot.resolve())
    print("Plot:", loglog_plot.resolve())
    print("Note: scaled n log2(n) is a visual reference, not a fitted model.")



if __name__ == "__main__":
    main()
```

B. Radix-2 FFT benchmark

Listing 3: FFT benchmark source.

```
"""Repeated runtime benchmark; writes CSV and an optional PNG plot."""
import argparse
import csv
import math
import statistics
import sys
import time
from pathlib import Path
sys.path.insert(0, str(Path(__file__).resolve().parents[1] / "src"))
from radix2_fft import fft


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--out", default="results")
    parser.add_argument("--min-power", type=int, default=8)
    parser.add_argument("--max-power", type=int, default=15)
    parser.add_argument("--repeats", type=int, default=7)
    parser.add_argument("--warmups", type=int, default=1)
    args = parser.parse_args()
    if args.min_power < 0 or args.max_power < args.min_power or args.repeats < 1 or
       args.warmups < 0:
        parser.error("invalid powers/repeats/warmups")
    out = Path(args.out)
    out.mkdir(parents=True, exist_ok=True)
    rows = []
    for power in range(args.min_power, args.max_power + 1):
        n = 2 ** power
        signal = [math.sin(0.013*i) + 0.25*math.cos(0.071*i) for i in range(n)]
        for _ in range(args.warmups):
            fft(signal)
        times = []
        for _ in range(args.repeats):
            start = time.perf_counter()
            fft(signal)
            times.append((time.perf_counter()-start)*1000)
        med = statistics.median(times)
        rows.append({"n": n, "n_log2_n": n*power, "repeats": args.repeats,
                     "median_ms": f"{med:.6f}", "min_ms": f"{min(times):.6f}",
                     "max_ms": f"{max(times):.6f}"})
        print(f"n={n:6d} median={med:10.4f} ms")
    csv_path = out / "fft_runtime.csv"
    with csv_path.open("w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=list(rows[0]))
        writer.writeheader()
        writer.writerows(rows)
    print(f"Wrote {csv_path}")
    try:
        import matplotlib.pyplot as plt
        ns = [int(r["n"]) for r in rows]
        medians = [float(r["median_ms"]) for r in rows]
        theoretical = [n*math.log2(n) for n in ns]
        scale = medians[0]/theoretical[0]
        plt.figure()
        plt.plot(ns, medians, marker="o", label="Measured median")
        plt.plot(ns, [v*scale for v in theoretical], marker="x", label="Scaled n
      log2(n)")
        plt.xlabel("Input size n")
        plt.ylabel("Runtime (ms)")
        plt.title("Radix-2 FFT runtime")
        plt.legend()
        plt.tight_layout()
        plot_path = out / "fft_runtime_vs_nlogn.png"
        plt.savefig(plot_path, dpi=160)
        plt.close()
        print(f"Wrote {plot_path}")
    except ImportError:
        print("Matplotlib not installed; CSV was written, plot skipped.")


if __name__ == "__main__":
    main()
```

APPENDIX B
REPRODUCIBILITY COMMANDS

Run the following commands from the repository root unless otherwise specified.

```
# Run greedy tests
python -m unittest discover \
    -s greedy_dsn/tests -p 'test_*.py' -v

# Check greedy optimality on random small
    instances
python greedy_dsn/tests/
    verify_greedy_optimality.py

# Run the greedy audit
python greedy_dsn/scripts/audit_satnet_v2.py \
    --problems greedy_dsn/data/problems.json \
    --maintenance greedy_dsn/data/maintenance.
    csv \
    --antenna DSS-34 \
    --out greedy_dsn/results/audit_v2

# Run the greedy scheduler
python greedy_dsn/src/greedy_satnet.py \
    --input greedy_dsn/results/audit_v2/
    candidate_intervals.csv \
    --out greedy_dsn/results/schedule
```

Run the FFT commands from the `divide_conquer_fft` directory:

```
# Run FFT tests
python -m unittest discover -s tests -v

# Run the FFT benchmark
python scripts/benchmark_fft.py \
    --out results \
    --min-power 8 \
    --max-power 15 \
    --repeats 7

# Run the real-data demonstration
python scripts/demo_real_data.py \
    --file data/105.mat \
    --sample-rate 12000 \
    --channel auto \
    --samples 4096 \
    --out results/real_data
```

REFERENCES

[1] Case Western Reserve University. *Bearing Data Center*. Available at: https://engineering.case.edu/bearingdatacenter.

[2] James W. Cooley and John W. Tukey. An algorithm for the machine calculation of complex Fourier series. *Mathematics of Computation*, 19(90):297–301, 1965. https://doi.org/10.1090/S0025-5718-1965-0178586-1.

[3] N. Dhamani, M. Johnston, and G. Lucena. Scheduling NASA's Deep Space Network: Priorities, Preferences, and Optimization. NASA Technical Reports Server, 2022. Available at: https://ntrs.nasa.gov/citations/20220001560.